

Week 2: Learning Values and Policies

Bellman equations, dynamic programming, and Monte Carlo learning

FYS4411 — Reinforcement Learning and Agentic AI
University of Oslo, Department of Physics

Week 2

Learning goals for Week 2

By the end of this week, you should be able to:

- derive the Bellman expectation equations from the return recursion;
- define optimal value functions and state the Bellman optimality equations;
- distinguish prediction from control;
- explain dynamic programming as a model-based use of Bellman equations;
- explain why sampled experience motivates model-free methods;
- implement and interpret Monte Carlo prediction and control;
- explain why Monte Carlo methods motivate temporal-difference learning.

1. Bellman equations: values as self-consistency relations
2. Optimal values: what does it mean to act optimally?
3. Prediction, control, and generalized policy iteration
4. Dynamic programming with a known model
5. Week 2 examples: GridWorld and FrozenLake
6. From model-based updates to sampled experience
7. Monte Carlo prediction, control, and Blackjack
8. Why temporal-difference methods come next

Week 1 introduced the agent–environment loop:

$$S_t \longrightarrow A_t \longrightarrow (R_{t+1}, S_{t+1}).$$

Week 2 asks a sharper question:

How do we find useful values and policies?

Takeaway. Values allow us to compare states and actions by their long-term consequences, not only by their immediate rewards.

One question, two computational settings

Model-based

The transition–reward kernel is available:

$$p(s', r \mid s, a).$$

We can compute expectations by summing over possible outcomes.

Model-free

The kernel is not available. We only observe samples:

$$(S_t, A_t, R_{t+1}, S_{t+1}).$$

We estimate values from interaction.

Discussion. Which of these two settings is closer to a laboratory experiment? Which is closer to a trusted simulator?

The three course pillars as RL problems

Quantum technology

actions are controls
or feedback
operations

uncertainty and
measurement outcomes

Molecular dynamics

actions change how
trajectories are
sampled

large state spaces and rare
transitions

Control / robotics

actions are applied in
real time

dynamics and sequential
decisions

The same mathematical language is used in all three:

state action reward policy return.

We use the notation from the course notes.

- State space: S .
- Action space: A .
- Reward space: $R \subseteq \mathbb{R}$.
- Admissible actions in state s : $A(s) \subseteq A$.
- Transition–reward kernel:

$$p(s', r \mid s, a) = P(S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a).$$

- Policy:

$$\pi(a \mid s) = P(A_t = a \mid S_t = s).$$

Return: the object being optimized

For a continuing task, the return is

$$G_t = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}.$$

For an episode ending at time T ,

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+k+1}.$$

The key recursive identity is

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Takeaway. This one-line recursion is the source of Bellman equations.

Value functions

For a fixed policy π , the state-value function is

$$v_{\pi}(s) = E_{\pi}[G_t \mid S_t = s].$$

The action-value function is

$$q_{\pi}(s, a) = E_{\pi}[G_t \mid S_t = s, A_t = a].$$

v_{π}

How good is this state if I follow π from now on?

q_{π}

How good is this action in this state if I follow π afterwards?

Derivation: start from the definition

Fix a policy π . Start with

$$v_{\pi}(s) = E_{\pi}[G_t \mid S_t = s].$$

Use the return recursion:

$$G_t = R_{t+1} + \gamma G_{t+1}.$$

Then

$$v_{\pi}(s) = E_{\pi}[R_{t+1} + \gamma G_{t+1} \mid S_t = s].$$

Takeaway. To evaluate this expectation, average first over the action chosen by π , then over the next state and reward produced by p .

Bellman expectation equation for v_π

If $S_t = s$, the policy chooses a with probability $\pi(a \mid s)$. Then the environment produces (s', r) with probability $p(s', r \mid s, a)$.

Therefore,

$$v_\pi(s) = \sum_{a \in A(s)} \pi(a \mid s) \sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a) [r + \gamma v_\pi(s')].$$

value now = reward now + discounted value later

Bellman expectation equation for q_π

For the action-value function,

$$q_\pi(s, a) = E_\pi[G_t \mid S_t = s, A_t = a].$$

The first action is already fixed, so we only average over the next state-reward pair:

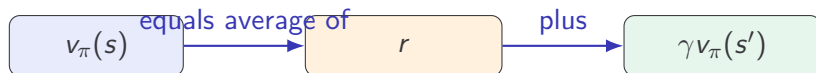
$$q_\pi(s, a) = \sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a) [r + \gamma v_\pi(s')].$$

Using

$$v_\pi(s') = \sum_{a' \in A(s')} \pi(a' \mid s') q_\pi(s', a'),$$

we can write a recursion entirely in terms of q_π .

Bellman equations as self-consistency



Bellman equations replace a long-horizon quantity G_t by a one-step recursive relation.

Takeaway. Most algorithms in this part of the course differ mainly in how they estimate or solve these recursive equations.

Optimal value functions

The value functions v_π and q_π evaluate a fixed policy. To compare policies, define

$$v_*(s) = \sup_{\pi} v_\pi(s), \quad q_*(s, a) = \sup_{\pi} q_\pi(s, a).$$

- $v_*(s)$ is the best achievable expected return from state s .
- $q_*(s, a)$ is the best achievable expected return after first taking action a in state s .

If q_* is known, an optimal deterministic policy is obtained by

$$\pi_*(s) \in \arg \max_{a \in A(s)} q_*(s, a).$$

Bellman optimality equation for v_*

Optimal behavior chooses the action with the best expected continuation value:

$$v_*(s) = \max_{a \in A(s)} \sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a) [r + \gamma v_*(s')].$$

This equation differs from the Bellman expectation equation in one place:

$$\sum_{a \in A(s)} \pi(a \mid s)(\cdots) \longrightarrow \max_{a \in A(s)} (\cdots).$$

Takeaway. Evaluation averages over the policy. Optimal control maximizes over actions.

Bellman optimality equation for q_*

The optimal action-value function satisfies

$$q_*(s, a) = \sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a) \left[r + \gamma \max_{a' \in A(s')} q_*(s', a') \right].$$

Interpretation:

- take action a now;
- observe r and s' ;
- behave optimally from the next state onward.

Discussion. Why is q_* especially useful when constructing a policy?

Prediction

The policy π is fixed. Estimate

$$v_{\pi}(s) \quad \text{or} \quad q_{\pi}(s, a).$$

Question:

How good is this policy?

Control

The policy is improved. Seek a policy with large expected return. Question:

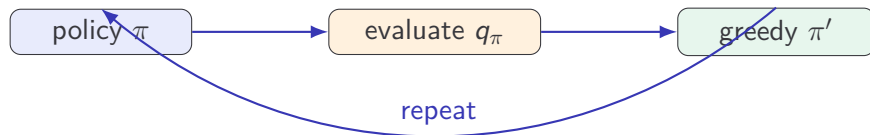
What should the agent do?

Many algorithms alternate between these two ideas.

Policy improvement

Suppose we have an estimate of $q_\pi(s, a)$ for a current policy π .
A natural improved deterministic policy is

$$\pi'(s) \in \arg \max_{a \in A(s)} q_\pi(s, a).$$



This pattern is called generalized policy iteration.

Dynamic programming: model-based Bellman updates

Dynamic programming assumes the environment model p is known. For a fixed policy, iterative policy evaluation starts with $v_0 : S \rightarrow \mathbb{R}$ and applies

$$v_{k+1}(s) = \sum_{a \in A(s)} \pi(a | s) \sum_{s' \in S} \sum_{r \in R} p(s', r | s, a) [r + \gamma v_k(s')].$$

Takeaway. Dynamic programming uses exact expectations over the known transition–reward kernel.

Policy iteration alternates two steps:

1. **Policy evaluation**: compute or approximate v_{π_k} .
2. **Policy improvement**: define

$$\pi_{k+1}(s) \in \arg \max_{a \in A(s)} \sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a) [r + \gamma v_{\pi_k}(s')] .$$

In compact form:

$$\pi_k \longrightarrow v_{\pi_k} \longrightarrow \pi_{k+1} .$$

Value iteration applies the Bellman optimality update directly:

$$v_{k+1}(s) = \max_{a \in A(s)} \sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a) [r + \gamma v_k(s')].$$

After convergence, a policy can be extracted by

$$\pi(s) \in \arg \max_{a \in A(s)} \sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a) [r + \gamma v(s')].$$

Takeaway. Policy iteration evaluates then improves. Value iteration mixes evaluation and improvement in one update.

Week 2 example: deterministic GridWorld

The notebook uses a small grid world:

- a 5×5 grid;
- black cells are obstacles;
- the green cell is the goal;
- moving has a small negative reward;
- reaching the goal gives reward 1 and terminates the episode.

For a deterministic transition, value iteration becomes

$$v(s) \leftarrow \max_a [r(s, a) + \gamma v(s')] .$$

GridWorld values and greedy policy

0.74 →	0.82 →	0.91 →	1.00 →	G
0.67 ↑		0.82 ↑		1.00 ↑
0.59 ↑		0.74 ↑	0.82 →	0.91 ↑
0.52 ↑	0.59 →	0.67 ↑		0.82 ↑
0.46 ↑		0.59 ↑	0.67 →	0.74 ↑

Values are the converged value iteration estimates; arrows are greedy actions.

Value iteration in code

The notebook loop mirrors the Bellman optimality update.

```
for iteration in range(max_iterations):
    delta = 0.0
    new_V = V.copy()

    for state in states:
        action_values = []
        for action in actions:
            next_state, reward, done = step(state, action)
            future_value = 0.0 if done else V[next_state]
            action_values.append(reward + gamma * future_value)

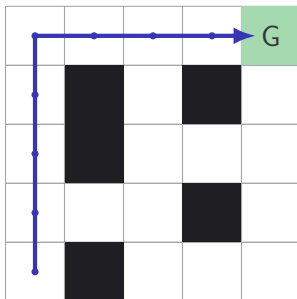
        new_V[state] = max(action_values)
        delta = max(delta, abs(new_V[state] - V[state]))

V = new_V
if delta < theta:
    break
```

Rollout under the greedy policy

Starting at $(4, 0)$, the greedy policy follows:

$(4, 0) \rightarrow (3, 0) \rightarrow (2, 0) \rightarrow (1, 0) \rightarrow (0, 0) \rightarrow (0, 1) \rightarrow (0, 2) \rightarrow (0, 3) \rightarrow (0, 4)$



The discounted return reported by the notebook is approximately 0.457.

Discussion: what made DP possible?

Discussion. Which part of the GridWorld example would fail if the function `step(state, action)` were not known?

Dynamic programming requires a way to compute or query the consequence of each action from each state.

In the stochastic case, this means access to probabilities such as

$$p(s', r \mid s, a).$$

Takeaway. Dynamic programming is not only an algorithmic idea. It is also an assumption about what information is available.

FrozenLake: stochastic model-based example

FrozenLake is a 4×4 episodic grid environment:

<i>S</i>	<i>F</i>	<i>F</i>	<i>F</i>
<i>F</i>	<i>H</i>	<i>F</i>	<i>H</i>
<i>F</i>	<i>F</i>	<i>F</i>	<i>H</i>
<i>H</i>	<i>F</i>	<i>F</i>	<i>G</i>

- *S*: start, *F*: frozen square, *H*: hole, *G*: goal.
- Reaching *G* gives reward 1 and terminates the episode.
- Falling into *H* terminates with reward 0.
- The slippery dynamics make transitions stochastic.

In the Gym environment, the model is stored as lists of outcomes:

$$\text{env.P}[s][a] = (\text{prob}, \text{next_state}, \text{reward}, \text{terminated}).$$

For example, from state 14 and action “right”, the notes give

$$\begin{aligned} \text{env.P}[14][2] = \{ & (1/3, 14, 0, \text{False}), \\ & (1/3, 15, 1, \text{True}), \\ & (1/3, 10, 0, \text{False}) \}. \end{aligned}$$

This is the implementation-level version of $p(s', r \mid s, a)$.

Finite-horizon dynamic programming

The notebook also uses a closed-form backward recursion.

Define

$V_t(s)$ = optimal expected reward from state s at time t .

Then for a finite horizon,

$$Q_t(s, a) = \sum_{s', r} p(s', r \mid s, a) [r + \mathbf{1}_{\text{not terminal}} V_{t+1}(s')] ,$$

$$V_t(s) = \max_a Q_t(s, a), \quad \pi_t(s) = \arg \max_a Q_t(s, a).$$

Takeaway. The policy can depend on time because the remaining horizon matters.

Closed-form agent: the essential loop

```
for t in range(max_episode_steps - 1, -1, -1):
    for s in range(state_n):
        for a in range(action_n):
            for p, next_s, r, d in P[s][a]:
                q[t, s, a] += p * (
                    r + (1. - float(d)) * v[t+1, next_s]
                )
            v[t, s] = q[t, s].max()
            pi[t, s] = q[t, s].argmax()
```

This is dynamic programming because the model P is used directly.

From model-based to model-free learning

In dynamic programming, the update contains exact expectations:

$$\sum_{s' \in S} \sum_{r \in R} p(s', r \mid s, a)(\dots).$$

In many settings, the agent cannot access $p(s', r \mid s, a)$ directly. It can only observe samples:

$$(S_t, A_t, R_{t+1}, S_{t+1}).$$

Takeaway. Model-free reinforcement learning replaces exact model-based expectations by empirical estimates from sampled trajectories.

Sampled experience

A sampled trajectory has the form

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \dots$$

Each transition sample

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

contains local information about what happened after one action.

It does *not* reveal the full kernel $p(s', r \mid s, a)$.



Monte Carlo prediction

Fix a policy π . The prediction problem is to estimate

$$v_{\pi}(s) = E_{\pi}[G_t \mid S_t = s].$$

Monte Carlo prediction estimates this expectation from sampled returns.

If state s has been visited $N(s)$ times and the observed returns are

$$G^{(1)}(s), G^{(2)}(s), \dots, G^{(N(s))}(s),$$

then

$$V(s) = \frac{1}{N(s)} \sum_{i=1}^{N(s)} G^{(i)}(s).$$

Takeaway. Monte Carlo prediction replaces an exact expectation by a sample average.

Why the sample average is reasonable

The state-value function is an expectation:

$$v_{\pi}(s) = E_{\pi}[G_t \mid S_t = s].$$

Monte Carlo produces sampled returns conditioned on visits to s .

The justification is the law of large numbers:

$$\frac{1}{N(s)} \sum_{i=1}^{N(s)} G^{(i)}(s) \longrightarrow E_{\pi}[G_t \mid S_t = s]$$

when sufficiently many relevant samples are collected.

Discussion. What can go wrong if a state is rarely visited?

First-visit and every-visit Monte Carlo

A state may appear more than once in the same episode.

First-visit

Use G_t only the first time $S_t = s$ appears in an episode.

$$S_j \neq s \quad \text{for all } 0 \leq j < t.$$

Every-visit

Use G_t for every time $S_t = s$ appears in an episode.

The same distinction applies to state-action pairs (s, a) .

Incremental sample average

Suppose $V_{n-1}(s)$ is the average after $n - 1$ observed returns, and a new return $G_n(s)$ is observed.

Then

$$V_n(s) = V_{n-1}(s) + \frac{1}{n} [G_n(s) - V_{n-1}(s)].$$

This updates the estimate by moving it toward the newly observed return.

Takeaway. This form reappears throughout reinforcement learning: old estimate plus step size times error.

Monte Carlo action-value prediction

For control, action values are especially useful:

$$q_{\pi}(s, a) = E_{\pi}[G_t \mid S_t = s, A_t = a].$$

Monte Carlo estimates q_{π} by averaging returns observed after visits to (s, a) :

$$Q(s, a) \approx q_{\pi}(s, a).$$

The incremental update is

$$Q_n(s, a) = Q_{n-1}(s, a) + \frac{1}{n} [G_n(s, a) - Q_{n-1}(s, a)].$$

Monte Carlo control alternates between two operations:

1. generate an episode using the current policy;
2. compute returns and update $Q(s, a)$;
3. improve the policy using the updated action values.

A greedy improvement would be

$$\pi'(s) \in \arg \max_{a \in A(s)} Q(s, a).$$

But a purely greedy policy may stop trying actions whose values are still uncertain.

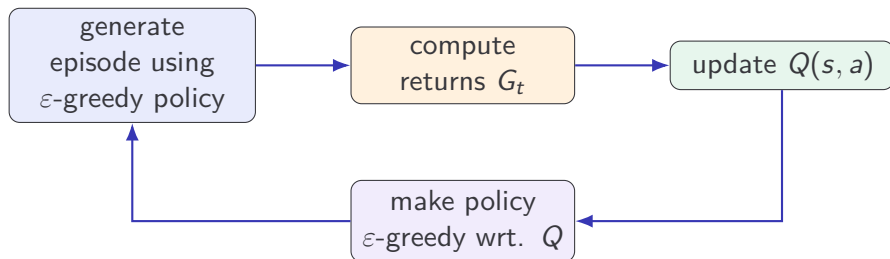
An ϵ -greedy policy usually chooses a greedy action, but sometimes explores.

For a finite action set $A(s)$ and a unique greedy action,

$$\pi(a \mid s) = \begin{cases} 1 - \epsilon + \frac{\epsilon}{|A(s)|}, & a \in \arg \max_{b \in A(s)} Q(s, b), \\ \frac{\epsilon}{|A(s)|}, & \text{otherwise.} \end{cases}$$

Takeaway. Exploration prevents the agent from committing too early to inaccurate estimates.

Monte Carlo control: algorithmic skeleton



$$Q(s, a) \leftarrow Q(s, a) + \alpha [G - Q(s, a)].$$

In on-policy Monte Carlo control, the same policy is used to:

- generate the episodes;
- define the value function being estimated;
- improve future behavior.

In the notebook discussion, the policy remains soft or ϵ -greedy, so all actions keep positive probability.

This matters because control requires exploration of state-action pairs.

Blackjack: why it is a good Monte Carlo example

Blackjack is naturally episodic:

- each game ends;
- the final reward is known at termination;
- the agent does not need transition probabilities for the card game;
- it can learn by generating many games and averaging observed returns.

At each decision point,

$$A = \{\text{stick}, \text{hit}\}.$$

In the implementation,

$$0 = \text{stick}, \quad 1 = \text{hit}.$$

Blackjack state and reward

A state is represented by

$$s = (\text{player sum}, \text{dealer card}, \text{usable ace}).$$

The reward is terminal:

$$R = \begin{cases} 1, & \text{player wins,} \\ 0, & \text{draw,} \\ -1, & \text{player loses.} \end{cases}$$

With $\gamma = 1$ and zero intermediate rewards, the final reward is the return for each visited state-action pair.

Blackjack: soft-policy Monte Carlo update

```
for state, action in state_actions:
    c[state][action] += 1.
    q[state][action] += (g - q[state][action]) / c[state][action]

a = q[state].argmax()
policy[state] = epsilon / 2.
policy[state][a] += 1. - epsilon
```

Mathematically,

$$q(s, a) \leftarrow q(s, a) + \frac{1}{c(s, a)} [G - q(s, a)].$$

For two actions, the greedy action receives probability

$$1 - \epsilon + \frac{\epsilon}{2},$$

and the other action receives $\epsilon/2$.

Blackjack: state-space shape

The notes use an array representation adapted to the observation:

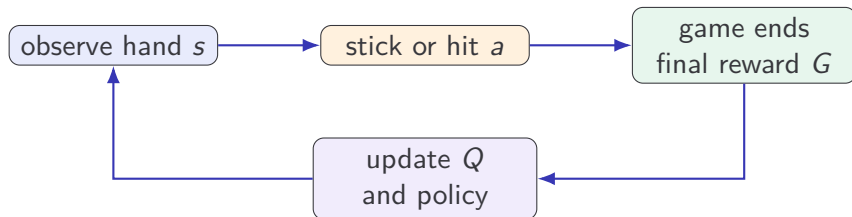
- player sum: $0, 1, \dots, 21$;
- dealer card: $1, \dots, 10$;
- usable ace: 0 or 1;
- action: 0 or 1.

Thus the policy and action-value arrays have shape

$(22, 11, 2, 2)$.

Takeaway. Tabular RL stores one value estimate for each state-action pair.

Blackjack as a complete Monte Carlo loop



Discussion. Where does the environment model appear in this loop?

From Blackjack back to physics

The Blackjack example is not physically important, but its structure is.

Monte Carlo learning applies when:

- an episode can be generated;
- a return can be computed after the episode;
- the model $p(s', r \mid s, a)$ is unavailable or not used;
- the goal is to estimate action values from repeated trials.

Takeaway. The transferable idea is not the card game. It is learning from complete sampled trajectories.

Pillar translation: quantum technology

Using only the RL notation, a quantum-control episode can be framed as:

- S_t : information available at step t ;
- A_t : a chosen control or feedback action;
- R_{t+1} : feedback from the environment;
- G_t : performance accumulated over the episode.

Monte Carlo becomes relevant if we can repeat episodes and update

$$Q(s, a) \leftarrow Q(s, a) + \alpha [G - Q(s, a)].$$

Discussion. What would make the episode terminate in a quantum-control experiment?

A molecular-dynamics task can be read as an episodic sampling problem:

- S_t : a discretized description of the current configuration;
- A_t : a sampling or control choice;
- R_{t+1} : feedback based on the outcome of the trajectory;
- G_t : total return of the trajectory.

Monte Carlo control would estimate which choices tend to produce better episode-level outcomes.

The mathematical requirement is the same as in Blackjack: repeated trajectories and returns.

A control or robotics task fits the same loop:

$$S_t \rightarrow A_t \rightarrow (R_{t+1}, S_{t+1}).$$

- S_t : position, velocity, or other available state information;
- A_t : the control action;
- R_{t+1} : feedback for progress, stability, or reaching a target;
- G_t : long-term success of the trajectory.

Monte Carlo is simple but waits for complete episodes. This is the bridge to temporal-difference learning.

Limits of Monte Carlo learning

Monte Carlo methods are model-free, but they have costs:

- updates normally wait until the episode ends;
- long episodes delay learning;
- sampled returns can have high variance;
- rare states or actions may be poorly estimated;
- continuing tasks do not naturally provide terminal returns.

Takeaway. Monte Carlo replaces exact expectations by complete observed returns. Temporal-difference learning will replace complete returns by one-step targets.

Motivation for temporal-difference learning

Monte Carlo target:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots .$$

Temporal-difference target:

$$R_{t+1} + \gamma V(S_{t+1}).$$

TD learning is still model-free, but it updates after one transition.

The basic TD error is

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

Monte Carlo versus TD: one-slide preview

Method	Target	Main feature
Monte Carlo	G_t	complete returns
TD prediction	$R_{t+1} + \gamma V(S_{t+1})$	one-step bootstrap
SARSA	$R_{t+1} + \gamma Q(S_{t+1}, A_{t+1})$	on-policy TD control
Q-learning	$R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a')$	off-policy TD control

This is the transition to next week.

Interactive checkpoint

1. In the Bellman expectation equation, where does the policy appear?
2. In the Bellman optimality equation, what replaces the policy average?
3. Why does dynamic programming require a model?
4. Why is Blackjack naturally suited to Monte Carlo learning?
5. Why might Monte Carlo be inefficient for long physical trajectories?

Week 2 has connected three ideas:

1. **Bellman equations**: value functions satisfy recursive self-consistency relations.
2. **Dynamic programming**: if $p(s', r | s, a)$ is known, we can solve these equations directly.
3. **Monte Carlo learning**: if the model is not used, we can estimate values from complete sampled returns.

Next week:

complete returns $\longrightarrow$ one-step bootstrapped targets.

Take-home message

known model $\Rightarrow$ dynamic programming

sampled episodes $\Rightarrow$ Monte Carlo learning

sampled transitions $\Rightarrow$ temporal-difference learning